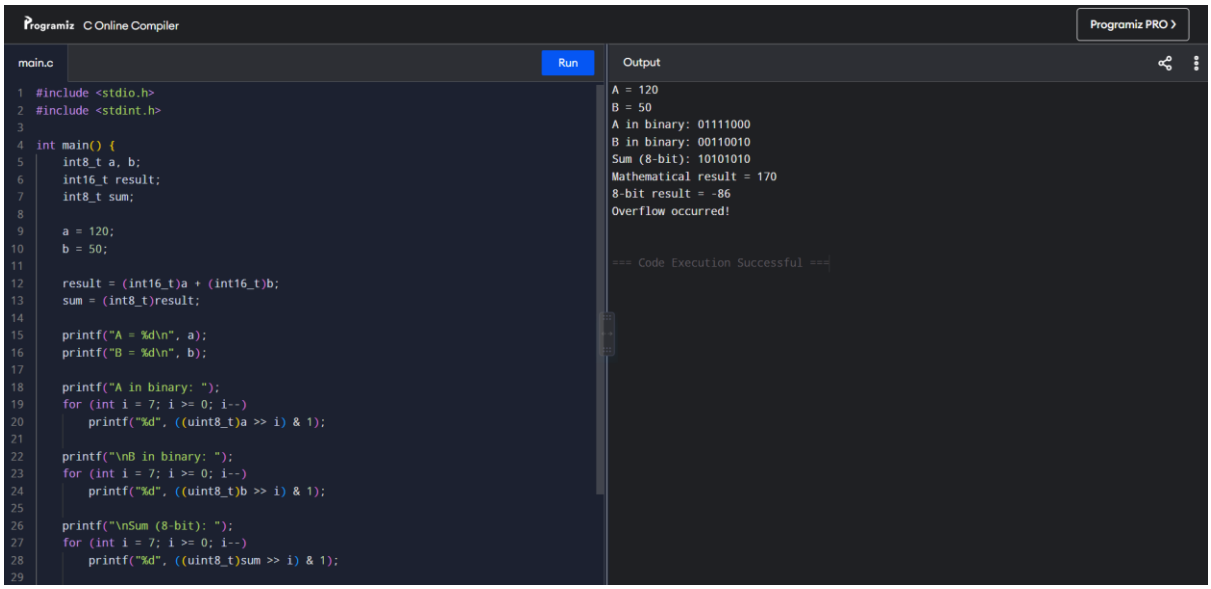
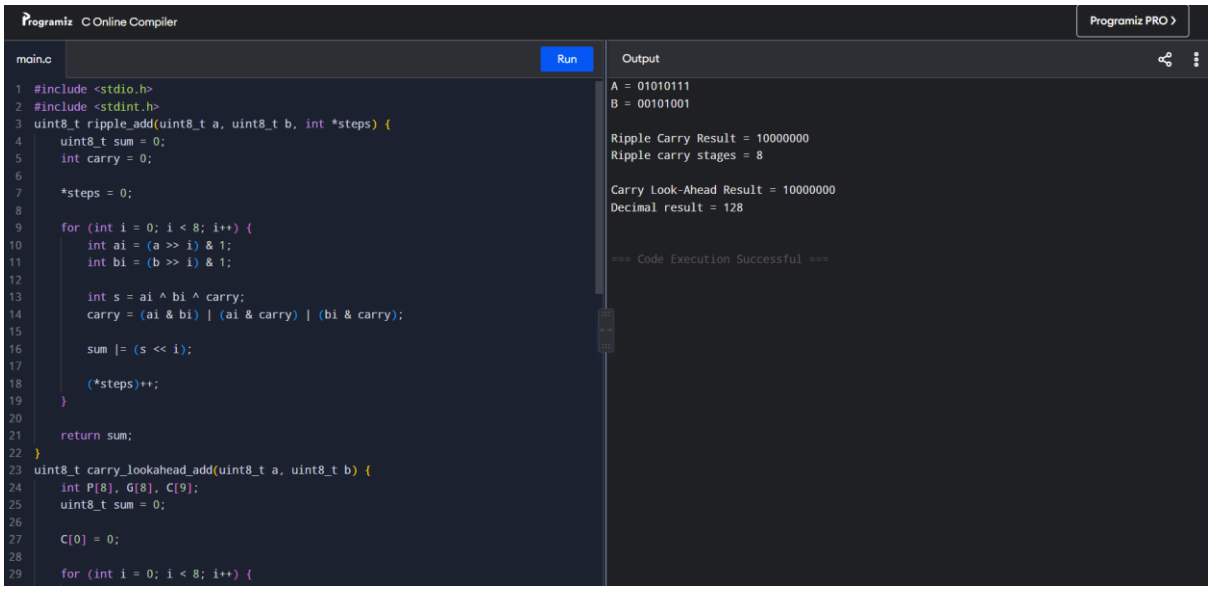
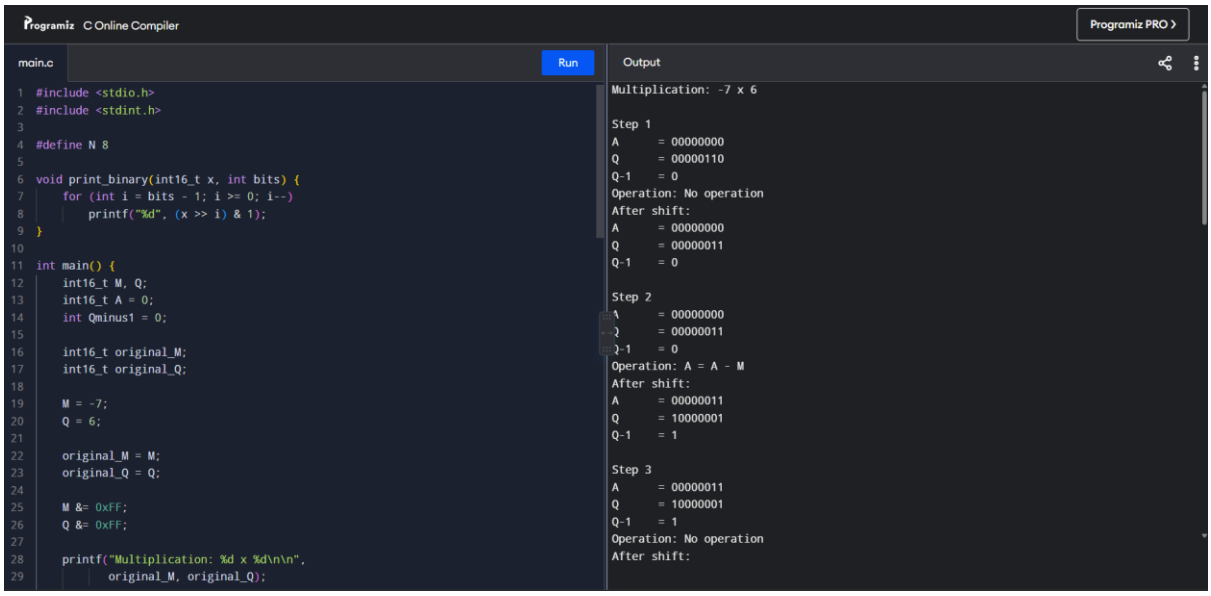
1.A program performing signed integer addition using 2’s complement produces incorrect results when adding large values (e.g., +120 and +50 in 8-bit representation). Debug the issue by analyzing the binary computation and identifying whether overflow has occurred. Propose a solution to correctly detect and handle overflow in such cases.

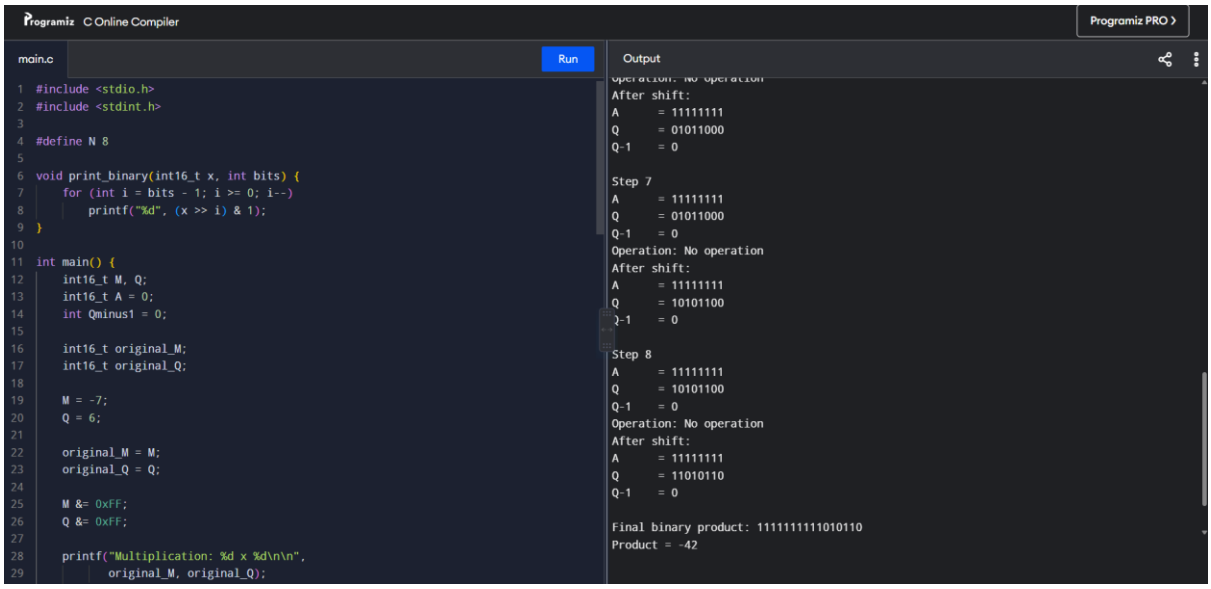



2.A processor using a ripple carry adder shows significant delay in executing arithmetic operations in a real-time system. Debug the performance bottleneck by examining how carry propagates through each bit. Suggest an optimized solution using a carry look-ahead adder, and explain how it reduces delay.



3.A multiplication unit implementing Booth's algorithm gives incorrect results for certain negative operands. Debug the step-by-step execution of the algorithm, focusing on bit-pair checking, arithmetic shifts, and sign extension. Identify the possible source of error and suggest corrections to ensure accurate signed multiplication.




4.An embedded system using the restoring division algorithm experiences inefficiency due to repeated restoration steps, leading to increased execution time. Debug the algorithm's workflow and identify where unnecessary operations occur. Propose an optimized approach using the non-restoring division method, explaining how it improves performance.

Programiz C Online Compiler

Programiz PRO >

main.c

Run

```c
#include <stdio.h>
#include <stdint.h>

void print_binary(int x, int bits) {
    for (int i = bits - 1; i >= 0; i--)
        printf("%d", (x >> i) & 1);
}

/* Non-restoring unsigned division */
void non_restoring_division(uint8_t dividend,
                            uint8_t divisor,
                            uint8_t *quotient,
                            uint8_t *remainder) {

    int A = 0;
    int Q = dividend;
    int M = divisor;

    printf("Dividend = %u\n", dividend);
    printf("Divisor  = %u\n\n", divisor);

    for (int i = 0; i < 8; i++) {

        /* Shift A and Q left */
        A = (A << 1) | ((Q >> 7) & 1);
        Q = (Q << 1) & 0xFF;

        printf("Step %d\n", i + 1);
```

Output

```
Dividend = 100
Divisor  = 7

Step 1
A = A - M
A = -7, Q = 200

Step 2
A = A + M
A = -6, Q = 144

Step 3
A = A + M
A = -4, Q = 32

Step 4
A = A + M
A = -1, Q = 64

Step 5
A = A + M
A = 5, Q = 129

Step 6
A = A - M
A = 4, Q = 3

Step 7
```

5. A scientific application using IEEE 754 single precision floating-point representation produces inaccurate results when adding very small and very large numbers. Debug the issue by analyzing exponent alignment, normalization, and rounding errors. Suggest optimization techniques, such as using double precision or algorithmic adjustments, to improve accuracy.

Programiz C Online Compiler

Programiz PRO >

main.c

Run

```c
#include <stdio.h>
#include <stdint.h>
#include <float.h>

void print_float_bits(float x) {
    union {
        float f;
        uint32_t u;
    } data;

    data.f = x;

    printf("Sign     : %u\n", (data.u >> 31) & 1);
    printf("Exponent : ");

    for (int i = 30; i >= 23; i--)
        printf("%u", (data.u >> i) & 1);

    printf("\nFraction : ");

    for (int i = 22; i >= 0; i--)
        printf("%u", (data.u >> i) & 1);

    printf("\n");
}

int main() {

    float large = 1.0e30f;
```

Output

```
IEEE 754 representation of large value:
Sign     : 0
Exponent : 11100010
Fraction : 10010011111001011001010

Single precision:
Large = 1.0000000150e+30
Small = 1.0000000000e+00
Result = 1.0000000150e+30
Small value was lost due to precision limitations.

Double precision:
Large = 1.000000000000000e+30
Small = 1.000000000000000e+00
Result = 1.000000000000000e+30

Float precision: 6 decimal digits
Double precision: 15 decimal digits


=== Code Execution Successful ===
```